

Specifiche Sintattiche del Linguaggio TriaC

1. Il Programma

In TriaC, un programma non è più un insieme disordinato di dichiarazioni globali e procedure. Per garantire la massima pulizia del generatore di codice LLVM, un programma è composto **esclusivamente da una sequenza di funzioni (func)**.

- Non sono ammesse variabili globali (fuori dalle funzioni) o procedure separate.
- **Nota Semantica:** All'interno del file sorgente deve esistere obbligatoriamente la funzione principale di ingresso, che deve avere la firma esatta:
`func main() -> int;`

2. Le Dichiarazioni di Variabili

TriaC abbandona i complessi terminatori come il backslash \ e le dichiarazioni multiple in linea che appesantirebbero il parser.

- In TriaC, ogni riga dichiara **una sola variabile** e deve terminare obbligatoriamente con il punto e virgola ;.
- La dichiarazione deve sempre specificare il tipo in modo esplicito.
- È consentito dichiarare una variabile senza inizializzarla (prenderà un valore nullo di default nel back-end) oppure inizializzarla immediatamente con un'espressione.

Esempi Corretti:

Python

```
var a: int;      # Dichiarazione semplice di intero
var x: double = 3.2; # Dichiarazione con inizializzazione immediata
var flag: boolean = true;
```

Esempi Errati:

Python

```
var b;          # ERRORE: Manca il tipo esplicito
var a, b: int;   # ERRORE: Dichiarazione multipla non consentita
var x: int = 3.2; # OK, ma genera un Warning Semantico di cast implicito (fptosi)
```

3. Tipi di Dato

I tipi utilizzabili in TriaC sono ridotti a tre tipi primitivi fondamentali per interfacciarsi in modo deterministico con LLVM IR :

- **int**: Mappato su un intero LLVM a 64 bit (**i64**).
- **double**: Mappato su un floating point LLVM a 64 bit (**double**).
- **boolean**: Mappato su un booleano LLVM a 1 bit (**i1**), con i letterali **true** e **false**.

Nota: Il tipo `string` non esiste come tipo di variabile manipolabile nel linguaggio. Le stringhe letterali sono ammesse solo come argomenti diretti per la funzione di stampa a schermo `print_string`.

4. Le Funzioni

Le funzioni in TriaC sono l'unico blocco d'esecuzione del linguaggio.

- Sono introdotte dalla parola chiave `func`.
- I parametri in ingresso sono specificati singolarmente, separati da virgola e tipizzati.
- **Semplificazione dei ritorni:** le funzioni in TriaC supportano **un solo tipo di ritorno esplicito** (es. `-> int`). Non sono supportati i ritorni multipli.
- **Layout-Sensitive:** Il corpo della funzione è preceduto dai due punti `:` ed è delimitato esclusivamente dall'**indentazione** (stile Python). Viene eliminata la parola chiave di chiusura `endfunc`.
- **Nota Semantica (Immutabilità):** I parametri formali dichiarati nella firma della funzione sono di sola lettura (`immutable`) all'interno del corpo e non possono essere mai riassegnati.
- Le dichiarazioni di variabili locali possono essere alternate liberamente alle istruzioni all'interno del corpo.

Esempio di Funzione in TriaC:

Python

```
func somma(a: int, b: int) -> int:  
    var c: int = 0;  
    c = a + b;  
    return c;
```

5. Le Assegnazioni

Un'assegnazione scrive un valore all'interno di una variabile precedentemente dichiarata.

- Utilizza l'operatore standard `=` al posto del vecchio `^=`, riducendo le ambiguità lessicali.
- È consentita solo l'assegnazione **singola**; le assegnazioni multiple simultanee sono abolite.
- Ogni assegnazione deve terminare con il punto e virgola `;`.

Esempi:

Python

```
x = 10;  
y = somma(4, 5);
```

6. Procedure (Abolite)

Le procedure **sono completamente abolite in TriaC** per standardizzare i blocchi di attivazione.

- Se si desidera scrivere una routine che non restituisce valori utili, si dichiara una normale funzione che restituisce un intero di cortesia (es. `-> int;`) e termina con `return 0;`.
- Viene eliminato il passaggio dei parametri per riferimento (`out / INOUT`) e l'uso del carattere `@` nelle chiamate, rendendo tutti i passaggi di parametri esclusivamente per valore e sicuri dal punto di vista degli effetti collaterali (side-effects) .

7. Input / Output (I/O)

TriaC elimina i complessi operatori grafici (`-->`, `-->!`, `<--`) e la sintassi di string-interpolation `$(...)` che renderebbero il parser inutilmente pesante e incline a conflitti . L'I/O in TriaC è gestito interamente tramite **chiamate a funzioni built-in** fornite dalla libreria runtime (collegate alla `libc` di sistema) :

- **Output:**
 - `print_int(val: int);`
 - `print_double(val: double);`
 - `print_string(val: string);` (stampa stringhe letterali costanti).
- **Input:**
 - `read_int() -> int;` (legge un intero da tastiera e lo restituisce)
 - `read_double() -> double;` (legge un decimale da tastiera e lo restituisce).

Esempio di I/O in TriaC:

```
Python
print_string("Inserisci un numero intero: ");
var x: int = read_int();
print_string("Hai inserito: ");
print_int(x);
```

8. Istruzioni (Statements)

Le istruzioni eseguibili in TriaC sono terminate da `;` e includono :

- **L'assegnazione** (`x = 5;`).
- **La chiamata a funzione** utilizzata come istruzione autonoma (`somma(1, 2);`).
- **L'istruzione di ritorno** (`return expr;`).
- **Il ciclo iterativo** `while (condizione);`.
- **La struttura condizionale** `if (condizione):` ed `else:`. *Nota:* Per mantenere pulito ed efficiente l'AST e la generazione dei blocchi base in LLVM, la variante `elseif` è stata rimossa; i condizionali a catena si realizzano semplicemente annidando costrutti `if-else` standard.

9. Espressioni (Expressions)

Le espressioni calcolano valori e includono :

- Le chiamate a funzione.
- Le costanti (numeri interi, decimali, `true`, `false`) .
- Gli identificatori di variabili.

- Le operazioni aritmetiche: `+`, `-`, `*`, `/`.
- Le operazioni logiche: **and**, **or**, **not** (stile Python, al posto dei operatori `&`, `||`, `!`) .
- Le operazioni di confronto: `==`, `!=`, `<`, `<=`, `>`, `>=` .